

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Modul 1: PENGANTAR MIKROKONTROLER

Nicholas Benaya - 5024241050

2026

1 Pendahuluan

1.1 Latar Belakang

Sistem tertanam (*embedded system*) merupakan bagian mendasar dari berbagai teknologi saat ini, mulai dari perangkat IoT, robotika, hingga otomasi industri. Penggunaan mikrokontroler 32-bit seperti STM32 dan ESP32 semakin banyak dipilih karena menawarkan performa komputasi yang lebih cepat, kapasitas memori yang lebih besar, dan periferal yang lebih lengkap dibandingkan mikrokontroler 8-bit konvensional. Dalam pengembangannya, pemilihan lingkungan kerja dan framework sangat memengaruhi alur kerja. Penggunaan Visual Studio Code bersama ekstensi PlatformIO memudahkan pengelolaan berbagai jenis mikrokontroler dalam satu tempat. Praktikan dapat memprogram menggunakan framework Arduino yang ringkas dan ramah pemula, maupun framework ESP-IDF yang lebih dekat dengan sistem operasi waktu nyata (FreeRTOS) untuk kontrol performa yang lebih optimal. Selain aspek software, pemahaman interfacing perangkat keras pada pin GPIO juga menjadi modal penting. Interaksi fisik antara tombol dan mikrokontroler rentan terhadap masalah seperti pin mengambang (*floating*) dan getaran mekanik (*bouncing*). Begitu pula dengan pembacaan sensor analog yang memerlukan pemahaman konversi ADC. Melalui praktikum Modul 1 ini, praktikan mempelajari dasar setup toolchain, proses upload program ke board STM32 dan ESP32, konfigurasi resistor pull-up/pull-down, teknik debouncing, serta pembacaan nilai analog secara langsung.

1.2 Dasar Teori

1.2.1 PlatformIO dan Visual Studio Code

PlatformIO adalah ekosistem pengembangan sistem tertanam yang terintegrasi langsung ke dalam Visual Studio Code. PlatformIO menyediakan compiler mandiri, manajemen library, dan pengaturan board secara otomatis. Seluruh konfigurasi proyek disimpan dalam file `platformio.ini`, sehingga pergantian jenis mikrokontroler maupun framework dapat dilakukan dengan mudah tanpa perlu berpindah aplikasi IDE.

1.2.2 STM32 Blackpill dan Framework Arduino

Papan STM32 Blackpill menggunakan chip STM32F401CCU6 berbasis inti ARM Cortex-M4 dengan kecepatan clock 84 MHz, Flash 256 KB, dan RAM 64 KB. Papan ini memiliki LED onboard di pin PC13 (aktif LOW) dan tombol pengguna di PA0. Blackpill tidak memiliki chip USB-to-Serial bawaan, sehingga pengunggahan program dilakukan menggunakan programmer eksternal ST-Link V2 melalui pin SWD (SWDIO, SWCLK, GND, dan 3.3V). Dengan framework Arduino (STM32duino), penulisan program menjadi praktis karena menggunakan fungsi standar Arduino dan mendukung pemanggilan nama pin fisik secara langsung.

1.2.3 ESP32 DevKit v1 dan Framework ESP-IDF

ESP32 DevKit v1 menggunakan prosesor dual-core Xtensa LX6 berkecepatan hingga 240 MHz dengan modul Wi-Fi dan Bluetooth terintegrasi. Papan ini sudah dilengkapi chip konverter USB-to-UART (seperti CP2102 atau CH340) dan rangkaian auto-reset, sehingga dapat langsung diprogram lewat

kabel USB biasa. ESP-IDF merupakan framework resmi dari Espressif yang berjalan di atas FreeRTOS. Alur program dimulai dari fungsi `app_main()` dan penundaan waktu menggunakan `vTaskDelay()`, yang menangguhkan task aktif tanpa membuang siklus prosesor secara percuma.

1.2.4 Konfigurasi Resistor Pull-up dan Pull-down

Pin GPIO masukan yang tidak terhubung ke tegangan tertentu akan berada pada kondisi mengambang (*floating*), sehingga nilainya berfluktuasi acak akibat derau listrik sekitar. Resistor pull-up dan pull-down digunakan untuk memastikan pin memiliki level logika yang pasti ketika saklar terbuka:

- **Pull-up**: Pin dihubungkan ke 3.3V melalui resistor. Saat tombol tidak ditekan, pin terbaca HIGH. Saat ditekan (terhubung ke GND), pin terbaca LOW.
- **Pull-down**: Pin dihubungkan ke GND melalui resistor. Saat tombol tidak ditekan, pin terbaca LOW. Saat ditekan (terhubung ke 3.3V), pin terbaca HIGH.

ESP32 memiliki resistor pull-up dan pull-down internal yang bisa diaktifkan lewat software, kecuali pada pin input-only (GPIO 34 sampai 39) yang tetap memerlukan resistor eksternal.

1.2.5 Fenomena Bouncing dan Debouncing

Saklar mekanik menggunakan lempengan logam lentur. Ketika tombol ditekan atau dilepas, kontak logam memantul sesaat (*bouncing*) dalam rentang 1 hingga 50 ms sebelum menempel rapat. Mikrokontroler yang membaca pin GPIO dengan cepat dapat menganggap pantulan ini sebagai penekanan berulang kali. Untuk mengatasinya, diterapkan teknik *debouncing*, baik secara software (menunggu sinyal stabil selama waktu tertentu menggunakan fungsi `millis()`) maupun hardware (menggunakan rangkaian filter RC dan Schmitt Trigger).

1.2.6 Analog-to-Digital Converter (ADC)

ADC berfungsi mengubah sinyal tegangan analog yang kontinu menjadi nilai digital diskret. ESP32 dilengkapi ADC internal beresolusi 12-bit, menghasilkan rentang nilai 0 hingga 4095 untuk tegangan masukan 0 sampai 3.3V. Pada praktikum ini, pembacaan analog diterapkan pada komponen LDR yang dirangkai sebagai pembagi tegangan bersama resistor 10kΩ, sehingga perubahan intensitas cahaya yang mengenai LDR dapat terbaca sebagai perubahan nilai digital pada mikrokontroler.

2 Tugas Pendahuluan

1. **Jelaskan perbedaan mendasar antara framework Arduino dan ESP-IDF dalam memprogram mikrokontroler, termasuk kelebihan dan kekurangan masing-masing.**

   Jawaban:

   Perbedaan mendasar antara framework Arduino dan ESP-IDF meliputi:

   - **Alur dan Struktur Program**: Arduino menggunakan struktur `setup()` untuk inisialisasi dan `loop()` untuk perulangan berkala. ESP-IDF berbasis FreeRTOS, di mana program berawal dari fungsi `app_main()` yang berjalan sebagai task independen.

- **Tingkat Abstraksi**: Arduino menyediakan abstraksi tingkat tinggi yang menyederhanakan akses register ke fungsi umum (`pinMode`, `digitalWrite`). Sebaliknya, ESP-IDF menggunakan driver perangkat keras resmi dan konfigurasi tingkat register yang lebih dekat ke hardware.
- **Manajemen Penundaan**: Fungsi `delay()` pada Arduino memblokir eksekusi CPU (*busy-waiting*), sedangkan `vTaskDelay()` pada ESP-IDF menangguhkan task aktif sehingga inti CPU dapat mengeksekusi task lain.

Kelebihan dan Kekurangan:

- **Framework Arduino**:
  - *Kelebihan*: Mudah dipelajari, mempercepat pembuatan purwarupa, dan didukung ekosistem pustaka komunitas yang sangat luas.
  - *Kekurangan*: Pemanfaatan memori kurang efisien dan akses ke fitur mendalam mikrokontroler (seperti alokasi task dual-core secara spesifik) terbatas.
- **Framework ESP-IDF**:
  - *Kelebihan*: Akses penuh ke seluruh fitur chip ESP32, kinerja lebih cepat, penggunaan memori optimal, dan siap untuk standar industri.
  - *Kekurangan*: Struktur kode lebih panjang dan rumit, serta kurva belajar lebih tinggi bagi pemula.

2. **Mengapa board STM32 Blackpill memerlukan programmer eksternal (ST-Link) untuk diflash, sedangkan ESP32 dapat diprogram langsung melalui kabel USB? Kaitkan jawaban Anda dengan keberadaan chip USB-to-Serial.**

   Jawaban:

   Perbedaan ini terjadi karena keberadaan chip konverter USB-to-Serial pada papan:

   - **ESP32 DevKit v1**: Papan ini sudah memiliki chip konverter USB-to-UART terintegrasi langsung di papannya (misalnya CP2102 atau CH340). Chip ini mengubah sinyal USB dari komputer menjadi sinyal serial UART (TX/RX) yang terhubung ke pin UART0 ESP32. Selain itu, terdapat sirkuit auto-reset berbasis sinyal DTR dan RTS yang secara otomatis mereset chip dan mengaktifkan mode bootloader saat flashing berlangsung.
   - **STM32 Blackpill**: Papan ini tidak memiliki chip konverter USB-to-UART terpisah untuk menghemat biaya dan ukuran. Port USB Type-C miliknya langsung menuju jalur USB OTG mikrokontroler. Tanpa bootloader khusus yang sudah terpasang, komputer tidak dapat mengenali port COM serial biasa. Oleh karena itu, pengunggahan program dan debugging dilakukan melalui jalur hardware SWD (pin SWDIO dan SWCLK) menggunakan alat programmer eksternal ST-Link V2.

3. **Jelaskan perbedaan kondisi logika (HIGH/LOW) pada konfigurasi pull-up dan pull-down resistor, baik saat tombol tidak ditekan maupun saat ditekan.**

   Jawaban:

   Resistor pull-up dan pull-down bertugas menjaga kondisi pin masukan agar tidak mengambang (*floating*). Perbedaan logikanya adalah sebagai berikut:

Konfigurasi	Saat Tombol Tidak Ditekan	Saat Tombol Ditekan
Pull-up	HIGH (tegangan mendekati 3.3V)	LOW (terhubung ke GND / 0V)
Pull-down	LOW (terhubung ke GND / 0V)	HIGH (terhubung ke 3.3V)

Tabel 1: Kondisi Logika Rangkaian Pull-up dan Pull-down

Pada konfigurasi pull-up, resistor menarik pin masukan ke tegangan 3.3V saat saklar terbuka. Saat tombol ditekan, jalur ke GND tertutup sehingga tegangan pin jatuh menjadi 0V. Pada konfigurasi pull-down, resistor menjaga pin tetap di ground (0V) saat terbuka, dan ketika ditekan, jalur ke 3.3V terhubung sehingga pin membaca logika HIGH.

4. **Jelaskan fenomena bouncing pada tombol mekanik: mengapa hal ini bisa terjadi, dan sebutkan minimal dua pendekatan untuk mengatasinya.**

   Jawaban:

   Penyebab Bouncing: Tombol mekanik bekerja menggunakan bilah logam lentur. Saat tombol ditekan atau dilepaskan, kontak logam tersebut memantul secara mikroskopis beberapa kali sebelum menempel sempurna. Hal ini menimbulkan osilasi sinyal digital HIGH dan LOW yang berlangsung cepat (biasanya 1 sampai 50 ms). Karena mikrokontroler membaca pin dengan frekuensi tinggi, satu kali penekanan tombol fisik dapat terbaca sebagai beberapa kali input yang tidak diinginkan. **Cara Mengatasinya:**

   (a) **Software Debouncing**: Menggunakan perhitungan waktu sistem (seperti fungsi `millis()` pada Arduino). Program mendeteksi perubahan sinyal, lalu menunggu selama durasi jeda tertentu (misalnya 20 sampai 50 ms). Sinyal baru dianggap sah apabila statusnya tetap stabil setelah durasi tersebut berlalu.
   (b) **Hardware Debouncing**: Menambahkan rangkaian filter low-pass RC (resistor dan kapasitor) pada jalur tombol untuk meredam lonjakan sinyal transien cepat, yang dapat dipadukan dengan gerbang Schmitt Trigger agar bentuk gelombang keluaran tetap berupa pulsa digital yang tegas.

5. **Bagaimana ADC (Analog-to-Digital Converter) mengubah sinyal analog menjadi nilai digital? Jelaskan pula pengaruh resolusi bit ADC terhadap presisi hasil pembacaan.**

   Jawaban:

   Proses Konversi ADC: ADC mengubah tegangan analog yang bersifat kontinu menjadi nilai angka digital diskret melalui tahapan:

   (a) *Sampling*: Mengukur nilai tegangan analog pada interval waktu tertentu.
   (b) *Holding*: Menahan nilai tegangan sampel menggunakan sirkuit kapasitor agar stabil selama proses konversi.
   (c) *Quantization*: Mengelompokkan tegangan yang ditahan ke dalam tingkatan diskret terdekat menggunakan komparator dan register aproksimasi (SAR).
   (d) *Encoding*: Mengubah tingkatan hasil kuantisasi menjadi kata biner berekstensi bit tertentu.

   Pengaruh Resolusi Bit: Resolusi bit (n) menentukan seberapa banyak tingkatan nilai biner yang dapat dihasilkan (2^n tingkat). Semakin tinggi resolusi bit, ukuran step tegangan terkecil per LSB

(*Least Significant Bit*) akan semakin kecil, dirumuskan dengan:

$$V_{LSB} = \frac{V_{ref}}{2^n - 1} \tag{1}$$

Dengan tegangan referensi 3.3V:

- ADC 10-bit menghasilkan 1024 tingkat dengan step tegangan ≈ 3.22 mV.
- ADC 12-bit (pada ESP32) menghasilkan 4096 tingkat dengan step tegangan ≈ 0.806 mV.

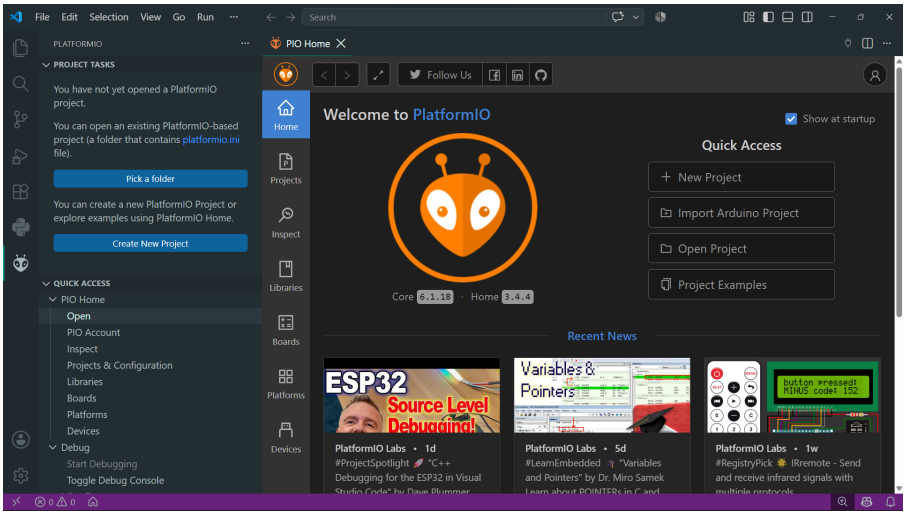
Dengan demikian, bit yang lebih tinggi menghasilkan resolusi pembacaan yang lebih halus dan kesalahan kuantisasi yang jauh lebih kecil.

6. **Lakukan set-up untuk tools: vscode, ekstensi platform-io (dengan framework esp-idf dan stm32), dan ekstensi serial-plotter pada laptop kalian, berikan screenshot bahwa laptop kalian sudah ter-set-up dengan baik.**

   Jawaban:

   Setup lingkungan pengembangan telah diselesaikan pada laptop dengan konfigurasi sebagai berikut:

   - Aplikasi Visual Studio Code telah terinstal.
   - Ekstensi PlatformIO IDE terpasang dan siap digunakan.
   - Platform `ststm32` (framework Arduino) untuk STM32 Blackpill dan `espressif32` (framework ESP-IDF dan Arduino) untuk ESP32 telah terpasang.
   - Ekstensi Serial Plotter pada VSCode telah ditambahkan untuk monitoring grafik serial.



Gambar 1: Tangkapan Layar Setup PlatformIO IDE dan Ekstensi Pendukung pada VSCode

Catatan: Tangkapan layar pada Gambar 1 menunjukkan antarmuka PlatformIO siap digunakan. Praktikan dapat mengganti file gambar ini dengan tangkapan layar langsung dari laptop masing-masing jika diperlukan.